

OpenCL vs C++ Backend Benchmark

Ten repeated runs of the full search and resample pipeline for both backends, on both datasets. All numbers below are measured wall time from repeated runs, not single samples.

00 Test system

All numbers in this report were measured on the same machine, back to back, nothing else competing for CPU or GPU.

CPU	Intel Core i7-10510U @ 1.80GHz • 4 cores / 8 threads • up to 4.9GHz boost
RAM	62GiB
GPU	Intel UHD Graphics (CometLake-U GT2) • OpenCL 3.0 NEO, driver 23.43.027642 • 24 compute units • max work-group 256
OS	Ubuntu 24.04.4 LTS • kernel 7.0.0-31-generic
Disk	NVMe SSD, 468GB

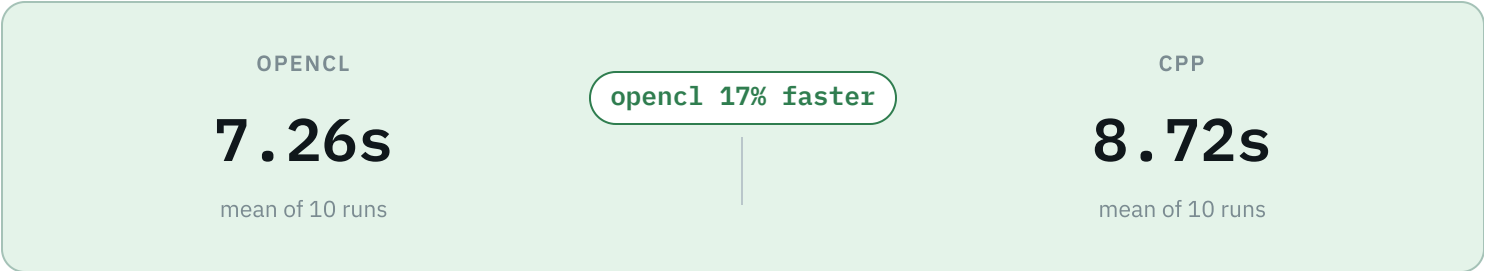
01 Result

Which backend wins depends on dataset size, and it flips. Ten runs each, taken back to back with nothing else competing for the GPU.

128x128, 500 projections



800x500, 1000 projections



DATASET	BACKEND	RUNS (S)								MEAN	MIN	MAX
128px	opencl	1.95	1.83	1.52	1.53	1.56	1.51	1.61	1.54	1.64s	1.51s	1.95s
		1.75	1.59									
128px	cpp	1.74	1.84	1.90	1.78	1.38	1.44	1.14	1.25	1.52s	1.14s	1.90s
		1.40	1.33									
512px	opencl	6.72	7.64	7.03	8.30	7.77	7.02	7.11	6.86	7.26s	6.72s	8.30s
		6.73	7.40									
512px	cpp	8.93	8.81	8.78	8.57	8.65	8.87	8.59	8.47	8.72s	8.47s	8.93s
		8.93	8.65									

i projection-center pipeline --backend X,timed with time.perf_counter() around each subprocess call, 10 repeats per backend per dataset.

02 **What was ruled out as the cause**

Resample kernel code

The C++ version and the Python version run the exact same kernel. Same math, same steps, nothing different there.

GPU buffer caching

Both backends set up their GPU memory once at the start and reuse it for every batch, instead of setting it up fresh each time. Both work the same way here, so this is not where the difference comes from.

03 **What batching bigger transfers changes**

Resample only, no search, 128px, 10 runs each, comparing the default `--batch-size 16` against one single transfer covering the whole dataset.

BACKEND	BATCH=16	BATCH=FULL	CHANGE
opencl	3.07s	2.30s	25% faster
cpp	2.31s	2.13s	8% faster

Doing fewer, bigger transfers speeds up both backends, but it helps opencl about three times more than it helps cpp. This closes part of the gap at the small dataset size, but not all of it.

04 **Where the two backends actually differ**

strace -c, aggregated over 10 runs each, resample only, 128px.

BACKEND	FUTEX TIME (10 RUNS)	SHARE OF TRACED TIME	CALLS
cpp	4.70s	52.6%	1539
opencl	40.10s	88.3%	2032

A futex is just a thread waiting its turn, not actual work being done. The opencl backend uses more internal threads behind the scenes than the cpp backend does, which shows up as more waiting. cpp also has to start up fresh as a new program on every run, while opencl stays running and ready.

05 **Open question**

Which backend wins really does depend on dataset size, and that part is confirmed. But there is still no clear single reason why it flips. The kernel code and the GPU memory setup are the same for both, and the threading difference found above does not fully explain it either. Figuring out the real cause would need a more focused test that measures raw processing speed at both sizes, not just total time.

06 Reference comparison

Latest measurement session (see the note below for how this compares to the session before it). Every implementation below is a full search+resample pipeline, 10 repeats each, one dataset per card.

128×128, 500 projections

IMPLEMENTATION	MEAN	MEDIAN	MIN	MAX	SPEEDUP VS REFERENCE	
reference	327.10s	325.78s	319.89s	336.09s	1×	
openc1	1.64s	1.57s	1.51s	1.95s	200×	
cpp	1.52s	1.42s	1.14s	1.90s	215×	
reference run 0	336.09s	openc1 run 0		1.95s	cpp run 0	1.74s
reference run 1	329.93s	openc1 run 1		1.83s	cpp run 1	1.84s
reference run 2	322.56s	openc1 run 2		1.52s	cpp run 2	1.90s
reference run 3	324.91s	openc1 run 3		1.53s	cpp run 3	1.78s
reference run 4	319.89s	openc1 run 4		1.56s	cpp run 4	1.38s
reference run 5	334.61s	openc1 run 5		1.51s	cpp run 5	1.44s
reference run 6	326.64s	openc1 run 6		1.61s	cpp run 6	1.14s
reference run 7	323.24s	openc1 run 7		1.54s	cpp run 7	1.25s
reference run 8	328.17s	openc1 run 8		1.75s	cpp run 8	1.40s
reference run 9	324.91s	openc1 run 9		1.59s	cpp run 9	1.33s

800×500, 1000 projections

IMPLEMENTATION	MEAN	MEDIAN	MIN	MAX	SPEEDUP VS REFERENCE
reference	5086.14s	5047.74s	4971.42s	5378.50s	1×
openc1	7.26s	7.07s	6.72s	8.30s	701×
cpp	8.72s	8.71s	8.47s	8.93s	583×
reference run 0	4981.44s	reference run 5	5014.68s	openc1 run 0	6.72s
reference run 1	4971.42s	reference run 6	5071.89s	openc1 run 1	7.64s
reference run 2	5257.49s	reference run 7	5050.91s	openc1 run 2	7.03s
reference run 3	4997.76s	reference run 8	5044.57s	openc1 run 3	8.30s
reference run 4	5378.50s	reference run 9	5092.70s	openc1 run 4	7.77s

opencl run 5	7.02s	cpp run 0	8.93s	cpp run 5	8.87s
opencl run 6	7.11s	cpp run 1	8.81s	cpp run 6	8.59s
opencl run 7	6.86s	cpp run 2	8.78s	cpp run 7	8.47s
opencl run 8	6.73s	cpp run 3	8.57s	cpp run 8	8.93s
opencl run 9	7.40s	cpp run 4	8.65s	cpp run 9	8.65s

Every mean above (reference, opencl, cpp) is 10 full pipeline runs from the same measurement session, each rep in its own subprocess, run in this order: reference, then opencl, then cpp.